

Project Title (Placeholder)

Camden Bergquist

January 30, 2026

Abstract

Table of contents

1	Abstract	4
2	Introduction	5
2.1	Background	6
3	The Data	9
3.1	Source	9
3.2	Variables	12
3.3	Observations	18
3.4	Cleaning	22
4	Exploratory Data Analysis	23
4.1	Interesting EDA 1 (Change name)	23
4.2	Interesting EDA 2 (Change name)	23
5	Methodology	24
5.1	Data Tranformations (if applicable)	24
5.2	Types of Models	24
5.2.1	Model 1 (Change to appropriate name)	24
5.2.2	Model 2 (Change to appropriate name)	24
5.2.3	Model 3 (Change to appropriate name)	24
5.2.4	Model 4 (Change to appropriate name)	25

5.2.5	Model 5 (Change to appropriate name)	25
5.2.6	Further Models	25
5.3	Method of model selection	25
6	Results	26
6.1	Model 1 (Change to appropriate name)	26
6.2	Model 2 (Change to appropriate name)	26
6.3	etc	26
6.4	Selected model	26
7	Discussion	27
7.1	Final model interpolation	27
7.2	Use of Model	27
7.3	Comparison	27
8	Furture Work	28
9	References	29
10	Appendix	30
10.1	Appendix A Graphs (Use appropriate name)	30
10.1.1	A.1 (Use appropriate name)	30
10.2	Appendix B Tables (Use appropriate name)	31
10.2.1	Appendix B.1 (Use appropriate name)	31

10.3 Code	31
---------------------	----

List of Figures

List of Tables

1	Core identifiers and categorical variables used across derived datasets. . . .	13
2	Player-month outcome and covariate variables (tables: <code>pm</code> , <code>pm_tc</code>).	13
3	Player-opening table variables (table: <code>pom_tc</code>).	15
4	Player time-use table variables (table: <code>ptm_tc</code>).	16
5	Metagame opening table variables (table: <code>mom_tc</code>).	17
6	Sample observation for player-period outcome summaries (tables: <code>pm</code> , <code>pm_tc</code>).	19
7	Sample observation for player-period time-use summaries (table: <code>ptm_tc</code>).	20
8	Sample observations for opening-level summaries: player repertoire entries (<code>pom_tc</code>) and population metagame cells (<code>mom_tc</code>).	21

Make sure every code chunk has `echo=FALSE` in its preamble unless you need to highlight the code for some reason in the paper.

1 Abstract

Complete this after all other sections have been completed.

2 Introduction

Competitive games have become a major source of large-scale behavioral data, enabling researchers to study strategic decision-making, skill development, and population-level trends using observational logs at scales unheard of compared to controlled studies ([Edelmann et al. 2020](#), [Watts 2014](#)). Chess is one of the most famous competitive games in the world, and the relatively recent growth of online platforms has created datasets with millions to billions of games played under standardized rules and well-defined competitive contexts (time controls, ratings, game outcomes, etc.) ([Watts 2014](#), [lichess.org 2026b](#)). One of the most widely used sources is the Lichess Open Database, which provides public monthly exports of rated games played on lichess.org in PGN format ([lichess.org 2026b](#), [Lichess contributors 2026](#)). These exports make it possible to study how strategy and performance vary across skill levels and time controls in a modern, online environment.

This project intends to use Lichess game data to investigate contemporary metagame behavior and player development. Here, ‘metagame’ refers to the population-level distribution of strategic choices and styles observed in practice – such as opening families, risk preferences, and game outcomes – within a defined time window and player population. In addition to describing how strategic behavior differs by rating and time control, the project will attempt to connect these strategic signatures to measures of player progression over time. Because chess is a repeatedly played, fully competitive setting with a mature rating infrastructure, it offers a natural testbed for quantifying skill and comparing behavior across different players ([lichess.org 2026a](#), [Glickman 2022](#)).

A central component of modern chess analytics is the rating system used to summarize player strength. The ELO rating system – a system originally developed for chess – mod-

els player performance probabilistically, and remains a foundational reference point for interpreting skill differences and expected outcomes in competitive settings (Elo 1978, International Chess Federation (FIDE) 2025). Lichess uses a modified version of this system called ‘Glicko-2’, which, in addition to rating, tracks uncertainty via rating deviation and volatility (lichess.org 2026a, Glickman 2022). Ratings are especially useful for observational studies because they provide a consistent scale by which to stratify players and construct longitudinal measures of improvement (Glickman 2022).

Overall, this project attempts to marry data science, computational social science, and strategic behavior: It treats online chess as a real-world environment where strategic choices and performance outcomes can be measured at scale and subsequently analyzed using statistical learning methods (Edelmann et al. 2020, Watts 2014).

2.1 Background

A growing research literature uses large online chess datasets to study human performance, decision-making, and strategic patterns. For example, Chowdhary et al. leverage an online chess dataset at the scale of over 100 million games to quantify performance dynamics and career-like trajectories, illustrating how large game archives support population-level behavioral measurement across skill levels (Chowdhary et al. 2023).

One important line of work focuses on time usage and decision quality under time constraints. Online chess platforms record both move sequences and clock information, which allows researchers to study how players allocate thinking time and how time pressure shapes decisions. Russek et al. analyze millions of Lichess games to relate move times to the estimated value of additional computation, and show that time allocation patterns vary systematically with playing strength and time control (Russek et al. 2025). These findings

support the idea that online chess is a rich setting for studying bounded rationality and performance under constraints, especially when comparing bullet, blitz, rapid, and classical formats ([Russek et al. 2025](#)).

Chess opening choice is also an active research topic. Because openings are discrete, labelable strategic choices that occur early in every game, they are well suited to population-level analysis. De Marzo and Servedio use online chess community data to quantify opening similarity and complexity, including representing openings in network form and studying how opening-related structure can be used to forecast future opening adoption ([De Marzo & Servedio 2023](#)). This motivates analyzing metagame development in terms of shifting opening popularity and success rates across time, player skill, and time controls ([De Marzo & Servedio 2023](#), [Chowdhary et al. 2023](#)).

Finally, there is an emerging interest in identifying player styles or archetypes from game logs. While traditional chess analysis often focuses on engine evaluation of positions, a complementary approach is to extract behavioral features from move sequences and outcomes (e.g., game length, opening selection, and other style proxies) and use statistical learning to characterize player types. For instance, McIlroy-Young et al. demonstrate that models trained on move sequences can capture individual-specific signatures (“style”) in chess decisions, providing evidence that player behavior is meaningfully distinguishable from game records alone ([McIlroy-Young et al. 2021](#)). This creates a bridge between strategic analysis (what players choose to do) and longitudinal outcomes (how players improve or persist over time), which aligns closely with the goals of this project.

Taken together, existing work suggests that large-scale online chess datasets support three relevant kinds of analysis: (1) descriptive metagame analysis of strategic choices across populations, (2) modeling how constraints like time pressure affect behavior and outcomes,

and (3) data-driven characterization of player types and their evolution ([Chowdhary et al. 2023](#), [Russek et al. 2025](#), [De Marzo & Servedio 2023](#), [McIlroy-Young et al. 2021](#)). This project intends to build on those themes by using Lichess data to analyze modern opening/style patterns and connect them to measures of player performance and development over time.

3 The Data

3.1 Source

Describe where this data comes from, why it was collected, who collected it, and how it was collected. A Citation may be useful here.

Lichess is the largest open-source chess server in the world, and releases free, monthly exports of their game data under the Creative Commons CCO license (lichess.org 2026b). The site requires a log of the game data in order to function while a game is being played, but holds onto it after the fact to allow users to analyze their games after playing them. Though Lichess offers a powerful API, I elected to download data directly from their website in `.pgn.zst` format. The PGN (portable game notation) format is a plaintext format for chess games, containing moves, headers like players, date, and event, and even optional annotation. The `.zst` file extension (sometimes also written as `zstd`) represents Zstandard compression, which is a modern compressor similar to gzip or xz that's commonly used because it's fast and compresses well; think of it as a variation on a ZIP or RAR file. Zstandard compression can be expected to shrink PGN files by 5 times on its faster, less-efficient settings, and even more on its slower ones.

For the purposes of this project, I downloaded 24 months of data spanning from January 2024 to December 2025, amounting to 677 gigabytes of raw data when compressed. Obviously, nearly 700 gigabytes of compressed data – 3~4 terabytes of uncompressed PGN data – is an insane amount to try to work with, capstone project or not. I have a powerful home computer, but that kind of scale requires something closer to a high-end server than a personal computer; it won't fly. Instead of tackling the problem head-on, I constructed a hybrid R/Python pipeline that downloads the data, deterministically samples and parses

it into small chunks (in the form of parquet files), and then builds a selection of smaller, ready-for-analysis datasets from those chunks. Individually, no one dataset has the 20+ variables required in the project outline, but since I’ve built multiple datasets for multiple different types of analysis, they collectively meet that benchmark

The download and parsing section of the pipeline handles the following:

- Reads a config file `pipeline.yml` that controls the scope and parameters of the pipeline’s operation.
- Downloads monthly Lichess ‘standard rated’ PGN archives for each month specified in the YAML.
- Decompresses each Zstandard file as a stream, scanning games one-by-one so it doesn’t run into RAM bottlenecks.
- Parses game headers first to decide whether a game is relevant before doing any heavy work (not all games are actually getting sampled, so it skips over the ones not being used).
 - Extracts core metadata (players, ratings, time control, result, termination, opening, timestamps).
 - Buckets each game into blitz/rapid/classical using the Lichess time-control convention.
- Applies filtering rules from the YAML, which in this case are:
 - Only include blitz/rapid/classical time controls.
 - Only include games from players in the 800–2600 rating range.
 - Drops unwanted terminations (aborted games).

- Uses deterministic hash-based sampling to create reproducible subsets without needing to store the whole population:
 - A meta sample keyed by `game_id` for population/metagame summaries, at a rate of 5/500 games.
 - A player cohort keyed by `username` for longitudinal/player tracking at a rate of 3/500 players.
 - A moves sample keyed by `game_id` for clock/time-per-move data at a rate of 5/10000 moves.
- Deterministically anonymizes usernames based on a stable, salted hash.
 - Cohort selection is computed on the real username before anonymization, so it remains reproducible.
- Writes month-partitioned parquet outputs for game-level data.

The dataset-building portion of the pipeline takes those outputs and, using DuckDB to read and write parquet files on the disk, builds seven datasets from them:

- Player-level longitudinal summary tables from `games_cohort`:
 - `player_month (pm)`: player * month, 249,761 observations.
 - `player_month_tc (pm_tc)`: player * month * time control, 320,869 observations.
- Player-opening ‘repertoire’ table from `games_cohort`:
 - `player_opening_month_tc (pom_tc)`: player * month * time control * opening, 7,167,580 observations.
- Metagame tables from `games_meta`:

- `meta_month_tc` (`mm_tc`): month * time control, 72 observations.
- `meta_opening_month_tc` (`mom_tc`): month * time control * opening, 130,672 observations.
- Time-usage tables from `moves_meta`:
 - `moves_month_tc` (`mvm_tc`) month * time control, 72 observations.
 - `player_time_month_tc` (`ptm_tc`): player * month * time control, 1,038,050 observations.

The pipeline logs session and version info for both R and Python and records checksums for downloaded raw files to help with reproducibility.

3.2 Variables

This project is working with multiple related-yet-distinct datasets. As such, it's very difficult to properly introduce the data without an according quantity of tables, which results in several pages worth of figures. My personal recommendation is to read through table 1, featured below, before moving straight to Section 3.3.¹ Most variable names are self-explanatory, so this section should largely be treated as a reference to skim through if a variable introduced later is confusing or unclear. Additionally, I elected to exclude tables `mm_tc` and `mvm_tc` from this section, as they're summary-level datasets that won't be explicitly modeled. Please refer to the <appendix placeholder; update when written> for information on them.

¹I'm considering consolidating the Variables and Observations sections into one, as I feel they're somewhat redundant, or else perhaps moving the Variable tables entirely into the appendix. I need to consult you for permission on that first, though.

Table 1: Core identifiers and categorical variables used across derived datasets.

Variable	Type	Description
<code>player</code>	categorical	Deterministic pseudonymous user ID (e.g., <code>u_...</code>). Appears in player-level tables (<code>pm</code> , <code>pm_tc</code> , <code>pom_tc</code> , <code>ptm_tc</code>).
<code>month</code>	categorical	Month of play as YYYY-MM. Primary time index for metagame and longitudinal analysis.
<code>tc_bucket</code>	categorical	Time control bucket: <code>blitz</code> , <code>rapid</code> , or <code>classical</code> . Used to stratify behavior by constraint level.
<code>eco</code>	categorical	ECO opening code (e.g., <code>C41</code>). Standardized opening taxonomy useful for grouping openings into families.
<code>opening</code>	categorical	Human-readable opening family name (e.g., <code>Philidor Defense</code>). Used in repertoire and metagame summaries.

Table 2: Player-month outcome and covariate variables (tables: `pm`, `pm_tc`).

Variable	Type	Description
<code>games</code>	numeric (count)	Number of games played in the month (and time-control bucket for <code>pm_tc</code>). Also a reliability/weighting signal.

Continued...

Variable	Type	Description
wins	numeric (count)	Count of wins in the period.
losses	numeric (count)	Count of losses in the period.
draws	numeric (count)	Count of draws in the period.
win_rate	numeric (proportion)	Key performance metric: wins divided by games. Natural response variable for performance models.
draw_rate	numeric (proportion)	Draws divided by games. Useful because draw propensity varies by skill, time control, and strategic choices.
avg_rating	numeric	Primary skill proxy: average of the player's pre-game rating across games in the period. Used for stratification and longitudinal change.
avg_opp_rating	numeric	Average opponent rating. Contextualizes performance and approximates strength of schedule.
avg_rating_diff	numeric	Average (player rating minus opponent rating). Summarizes typical matchup difficulty.
avg_moves	numeric	Average game length (moves). Proxy for style/complexity.
opening_diversity	numeric (count)	Number of distinct openings used in the period. Proxy for repertoire breadth.

Continued...

Variable	Type	Description
top_opening	categorical	Most frequently played opening during the period (ties broken deterministically).

Table 3: Player-opening table variables (table: `pom_tc`).

Variable	Type	Description
games	numeric (count)	Number of games the player played with the given opening in the month/time-control bucket. Used as a frequency weight.
win_rate	numeric (proportion)	Opening-specific win rate for the player and period. Used to study effectiveness conditional on opening.
draw_rate	numeric (proportion)	Opening-specific draw rate. Helps characterize risk profile and style.
avg_rating	numeric	Average player rating in games where the opening was played. Enables comparison of opening choice across skill.
avg_rating_diff	numeric	Average rating difference in games where the opening was played. Captures whether openings are used in tougher/easier matchups.

Table 4: Player time-use table variables (table: `ptm_tc`).

Variable	Type	Description
<code>move_rows</code>	numeric (count)	Number of move observations contributing to the summary (after sampling and filtering). Reliability signal.
<code>games</code>	numeric (count)	Number of distinct games contributing move records for the player-month-time-control group.
<code>avg_time_spent</code>	numeric (seconds)	Mean time spent per move. Captures typical deliberation time.
<code>median_time_spent</code>	numeric (seconds)	Median time spent per move; robust to outliers and occasional long thinks.
<code>p90_time_spent</code>	numeric (seconds)	90th percentile of time spent per move. Measures tendency for occasional deep computation.
<code>avg_clock_after</code>	numeric (seconds)	Mean remaining clock time after moves. Proxy for time management and pacing.
<code>low_time_rate</code>	numeric (proportion)	Fraction of moves under a low-time threshold. Proxy for time trouble frequency.
<code>avg_rating</code>	numeric	Average player rating across move records in the group; allows stratification by skill.
<code>avg_rating_diff</code>	numeric	Average rating difference across the move sample; controls for opponent strength effects.

Continued...

Variable	Type	Description
avg_time_ opening	numeric (seconds)	Average time spent per move during the opening phase (if phase segmentation is available; game phases are defined by chess-engine analysis).
avg_time_ middlegame	numeric (seconds)	Average time spent per move during middlegame phase (may be missing for very short games).
avg_time_ endgame	numeric (seconds)	Average time spent per move during endgame phase (often missing because many games never reach endgame).

Table 5: Metagame opening table variables (table: `mom_tc`).

Variable	Type	Description
games	numeric (count)	Number of sampled games for the opening within month and time control. Precision/support indicator.
share	numeric (proportion)	Core metagame measure: fraction of games using the opening within a month/time-control bucket. Tracks adoption shifts.
white_win_rate	numeric (proportion)	Fraction of games where White wins. Used for opening-level success patterns and color asymmetries.

Continued...

Variable	Type	Description
<code>black_win_rate</code>	numeric (proportion)	Fraction of games where Black wins. Complements White win rate.
<code>draw_rate</code>	numeric (proportion)	Fraction of games drawn. Draw propensity varies by opening and time control.
<code>upset_rate</code>	numeric (proportion)	Fraction of games where the lower-rated player wins (pipeline definition). Proxy for volatility/competitiveness.
<code>avg_abs_rating_diff</code>	numeric	Mean absolute rating gap between players in the opening/month/tc group. Context for outcome rates.
<code>avg_moves</code>	numeric	Mean game length for the opening/month/tc group. Proxy for complexity and style at population level.

3.3 Observations

Each derived dataset is organized at a different unit of analysis (e.g., player-by-month or opening-by-month). In all cases, the raw unit of data is a rated chess game from the Lichess public database, but the pipeline aggregates games (and, for time-use, moves) into analysis-ready summaries. Because many variables overlap across tables, the observation structure can be summarized with a small set of representative row types. The examples below demonstrate the core units: (1) player-period summaries (with optional time-use extensions), (2) player opening-level summaries, and (3) population-level metagame summaries for openings.

Table 6: Sample observation for player-period outcome summaries (tables: `pm`, `pm_tc`).

Field	Example Value/Meaning
<code>player</code>	<code>u_0001cf80083d0517</code> (stable pseudonymous identifier)
<code>month</code>	<code>2024-12</code> (calendar month)
<code>tc_bucket</code>	<code>rapid</code> (present in <code>pm_tc</code> ; omitted in <code>pm</code>)
<code>games</code>	5 (total games in this player-month-(tc) group)
<code>wins</code>	4
<code>losses</code>	0
<code>draws</code>	1
<code>win_rate</code>	0.80 (wins / games)
<code>draw_rate</code>	0.20 (draws / games)
<code>avg_rating</code>	1544.0 (mean pre-game rating across games)
<code>avg_opp_rating</code>	1503.2 (mean opponent rating across games)
<code>avg_rating_diff</code>	40.8 (mean rating difference: player minus opponent)
<code>avg_moves</code>	40.4 (mean game length in moves)
<code>opening_</code>	4 (distinct openings played in the group)
<code>diversity</code>	
<code>top_opening</code>	Ponziani Opening: Jaenisch Counterattack (most frequent opening)

Table 7: Sample observation for player-period time-use summaries (table: `ptm_tc`).

Field	Example Value/Meaning
<code>player</code>	<code>u_00004578077e9fa4</code> (stable pseudonymous identifier)
<code>month</code>	<code>2024-08</code>
<code>tc_bucket</code>	<code>blitz</code>
<code>move_rows</code>	70 (number of move records contributing to this summary)
<code>games</code>	1 (distinct games represented among the sampled moves)
<code>avg_time_spent</code>	1.66 seconds (mean time spent per move)
<code>median_time_spent</code>	1.0 seconds (median time spent per move)
<code>p90_time_spent</code>	3.1 seconds (90th percentile time spent per move)
<code>avg_clock_after</code>	122.5 seconds (mean remaining clock after moves)
<code>low_time_rate</code>	0.00 (share of moves flagged as low-time)
<code>avg_rating</code>	1677.0 (mean pre-move rating across sampled moves)
<code>avg_rating_diff</code>	11.0 (mean rating difference across sampled moves)
<code>avg_time_opening</code>	0.9 seconds (mean time spent in opening phase)
<code>avg_time_middlegame</code>	2.2 seconds (mean time spent in middlegame phase)
<code>avg_time_endgame</code>	1.64 seconds (mean time spent in endgame phase; may be missing)

Table 8: Sample observations for opening-level summaries: player repertoire entries (`pom_tc`) and population metagame cells (`mom_tc`).

Field	Example Value/Meaning
Player opening summary (table: <code>pom_tc</code>)	
<code>player</code>	<code>u_0001cf80083d0517</code>
<code>month</code>	<code>2024-11</code>
<code>tc_bucket</code>	<code>blitz</code>
<code>eco</code>	<code>B13</code>
<code>opening</code>	<code>Caro-Kann Defense: Exchange Variation</code>
<code>games</code>	4 (games where this player used the opening in this month/tc)
<code>win_rate</code>	0.50 (opening-conditional win rate for the player)
<code>draw_rate</code>	0.00 (opening-conditional draw rate for the player)
<code>avg_rating</code>	1965.3 (mean rating in games using this opening)
<code>avg_rating_diff</code>	12.0 (mean rating difference in those games)
Population opening summary (table: <code>mom_tc</code>)	
<code>month</code>	<code>2024-01</code>
<code>tc_bucket</code>	<code>blitz</code>
<code>eco</code>	<code>C41</code>
<code>opening</code>	<code>Philidor Defense</code>
<code>games</code>	8,825 (sampled games using this opening in the month/tc group)
<code>share</code>	0.0189 (opening share of the month/tc metagame)
<code>white_win_rate</code>	0.512 (fraction of games White wins)

Continued...

Field	Example Value/Meaning
<code>black_win_rate</code>	0.442 (fraction of games Black wins)
<code>draw_rate</code>	0.0466 (fraction drawn)
<code>upset_rate</code>	0.436 (fraction where lower-rated player wins)
<code>avg_abs_rating_diff</code>	50.2 (mean rating gap magnitude)
<code>avg_moves</code>	35.5 (mean game length)

Together, these observation types support longitudinal player performance modeling (`pm`, `pm_tc`), repertoire/style measurement from opening choice (`pom_tc`), metagame dynamics via opening adoption and outcomes (`mom_tc`), and bounded-rationality/time-management analyses using move-time features (`ptm_tc`).

3.4 Cleaning

A large amount of cleaning was performed during the pipeline’s parsing steps. The filtering and deterministic sampling was already touched upon in `#sec-source`, but additional standardization and feature construction include:

- `TimeControl` tag parsed into:
 - `initial` (seconds).
 - `increment` (seconds).
 - `tc_bucket` (`< 180` = bullet, `< 600` = blitz, `< 1800` = rapid, else classical),
- Converted PGN result strings into:

- `winner` (white/black/draw).
 - `score_white` (1/0.5/0).
- Computed upset indicators using ratings:
 - `rating_diff = white_rating - black_rating`.
 - `abs_rating_diff`
 - `is_upset`
- Game-length variables:
 - Computed `piles` and `moves` from move sequences (used to construct final datasets).

The information was then restructured and aggregated into a tidy format.

4 Exploratory Data Analysis

Show the initial graphs and basic analysis methods used to determine your methodology

4.1 Interesting EDA 1 (Change name)

Show some graph or numerical summary that imparts an interesting idea about the data set

4.2 Interesting EDA 2 (Change name)

Show some graph or numerical summary that imparts an interesting idea about the data set

5 Methodology

Describe the methods used to analyze the data. Do not discuss the analysis itself. Make sure to discuss any train/test data splitting here.

5.1 Data Transformations (if applicable)

Describe any transformation you did to the data to assist in the analyzing of the data such as creating dummy variables.

5.2 Types of Models

Describe how many different models you used and why.

5.2.1 Model 1 (Change to appropriate name)

Describe why you used model type one. You do not need to include a new section for every iteration of the same model type. For example you do not need 4 sections if you have 4 linear models with 1, 2, 3, and 4 independent variables.

5.2.2 Model 2 (Change to appropriate name)

Describe why you used model type two.

5.2.3 Model 3 (Change to appropriate name)

Describe why you used model type three.

5.2.4 Model 4 (Change to appropriate name)

Describe why you used model type four.

5.2.5 Model 5 (Change to appropriate name)

Describe why you used model type five.

5.2.6 Further Models

If using more than 5 types of models, use this section to quickly discuss them here. These should only be models that you do not expect to be great.

5.3 Method of model selection

Describe how you determine which model you choose at the end. Should also describe why you chose this method.

6 Results

Describe the results that you received from the methods described earlier.

6.1 Model 1 (Change to appropriate name)

Show the results for model type one. At least one fully written out model should be displayed for each model type.

6.2 Model 2 (Change to appropriate name)

Show the results for model type two.

6.3 etc

6.4 Selected model

Display the table that shows the model selection. If using more than 10 models only show the top 10 performing in the table, the full table should be in the appendix. The final model should be fully displayed.

7 Discussion

7.1 Final model interpolation

Discuss what information the selected model provides that may be new and interesting

7.2 Use of Model

Describe how this model might be used by others.

7.3 Comparison

Compare the things learned from this selected model to the previous information.

8 Furture Work

Discuss what work could be done with this model to make it better.

9 References

List the reference used for your paper. This should include anything cited in your background section and the source of your data.

10 Appendix

10.1 Appendix A Graphs (Use appropriate name)

10.1.1 A.1 (Use appropriate name)

If showing multiple graphs make sure to index them by 0.1 each

10.2 Appendix B Tables (Use appropriate name)

10.2.1 Appendix B.1 (Use appropriate name)

If showing multiple tables make sure to index them by 0.1 each

10.3 Code

```
library(tidyverse)

# Reason for library ...

#library(...)

# Reason for library ...

#library(...)

# Set seed is used because question ... uses a random generator

set.seed(123)

# Load cleaned data here. If cleaning was required make sure to have a separate r file .

# Use this code to display useful graphs that do not belong in the body of the paper.

# For example use this area for model validation graphs like residual versus fit.

# Use this area to show results that are in table format

# Write your cleaning code here
```

References

Chowdhary, S., Iacopini, I. & Battiston, F. (2023), ‘Quantifying human performance in chess’, *Scientific Reports* **13**, 2113.

URL: <https://doi.org/10.1038/s41598-023-27735-9>

De Marzo, G. & Servedio, V. D. P. (2023), ‘Quantifying the complexity and similarity of chess openings using online chess community data’, *Scientific Reports* **13**, 5327.

URL: <https://doi.org/10.1038/s41598-023-31658-w>

Edelmann, A., Wolff, T., Montagne, D. & Bail, C. A. (2020), ‘Computational social science and sociology’, *Annual Review of Sociology* **46**, 61–81.

URL: <https://doi.org/10.1146/annurev-soc-121919-054621>

Elo, A. E. (1978), *The Rating of Chessplayers, Past and Present*, Arco Publishing, New York.

Glickman, M. E. (2022), The Glicko-2 system for rating players in head-to-head competition, Technical report, Boston University. Technical description of Glicko-2 parameters (rating, rating deviation, volatility) and update procedure.

URL: <https://www.glicko.net/ratings/glicko2desc.pdf>

International Chess Federation (FIDE) (2025), ‘Anniversary of arpad elo – rating system that changed chess world’. Describes the origins of the Elo system for chess and its implementation history.

URL: <https://www.fide.com/anniversary-of-arpad-elo-rating-system-that-changed-chess-world/>

Lichess contributors (2026), ‘lichess-org/database: Public exports of all rated games, played on lichess.org’. Repository describing the export of the Lichess game database in standard PGN format.

URL: <https://github.com/lichess-org/database>

lichess.org (2026a), ‘Chess rating systems’. Explains that Lichess uses Glicko-2 and notes platform-specific adjustments.

URL: <https://lichess.org/page/rating-systems>

lichess.org (2026b), ‘Lichess open database’. Public monthly exports of rated games in PGN format; files are monthly (not cumulative) and distributed as .zst-compressed PGN.

URL: <https://database.lichess.org/>

McIlroy-Young, R., Wang, R., Sen, S., Kleinberg, J. & Anderson, A. (2021), Detecting individual decision-making style: Exploring behavioral stylometry in chess, *in* ‘Advances in Neural Information Processing Systems’. NeurIPS 2021 paper on modeling and identifying player styles from chess move data.

URL: https://proceedings.neurips.cc/paper_files/paper/2021/hash/ccf8111910291ba472b385e9c5f59099-Abstract.html

Russek, E. M., Acosta-Kane, D., van Opheusden, B., Mattar, M. G. & Griffiths, T. L. (2025), ‘Time spent thinking in online chess reflects the value of computation’, *Cognitive Science* **49**(10), e70119.

URL: <https://doi.org/10.1111/cogs.70119>

Watts, D. J. (2014), Computational social science: Exciting progress and future directions, *in* ‘Frontiers of Engineering: Reports on Leading-Edge Engineering from the 2013 Symposium’, National Academies Press, Washington, DC. Discusses the rise of large-scale behavioral data for studying social and behavioral phenomena.

URL: <https://doi.org/10.17226/18558>